MarkGo Subscription Build Guide

Build order, dummy payment, and MVP checklist

This guide covers adding subscription plans (Free, Monthly, Yearly) between the platform (Super Admin) and each company (Company Admin), using a dummy payment for now. Everything is kept simple and MVP-only.

Big idea: a **Plan** is what the Super Admin sells (name, price, duration). A **Subscription** is what a company bought (which plan, start date, end date, price paid). Editing a plan never changes an existing subscription, so companies on an old plan keep working until their time ends.

1. Suggested build order

Do the steps in this order. The most important rule: **turn on blocking (login check, plan limits) only at the very end**, after every screen exists. Otherwise companies get blocked with nowhere to go.

1 Prepare

- Back up your database.
- Decide the open questions (see Part 3, items 4 to 7).
- Add two settings to your env file: `PAYMENT_MODE=dummy` and `ENFORCE_SUBSCRIPTION=false`.

Test before moving on: the backup file opens, and the app still starts.

2 Schema and migration

- Update **Plan** (billing cycle, duration, extra limits) and **SubscriptionPayment** (link to subscription, method).
- Add the new **Subscription** model.
- Add `PENDING_PAYMENT` to the company status list and link Company to subscriptions.
- Run the migration.

Test before moving on: the migration runs with no error and the existing company admin and super admin pages still load.

3 Seed plans and existing companies

- Create three plans: Free, Monthly, Yearly (price, duration, limits). Free has small limits, paid plans have no limit.
- Run a one-time script that gives **every existing company** a subscription (for example Free, or a long trial).

Test before moving on: every company in the database has one active subscription. Do not skip this, or turning on the login check later will lock everyone out.

4 Backend part A (no blocking yet)

- Public API: list active plans.
- Register API: accept a plan. Free plan makes the company active at once. A paid plan creates the company as `PENDING_PAYMENT` and returns a payment record.
- Dummy payment service (see Part 2) and its confirm API.
- Company admin APIs: current subscription, history, buy or renew, start free.
- Super admin APIs: create and edit plans (price, duration, free plan active or inactive), set a company active or inactive, register a company with a plan (can mark payment as MANUAL), assign a plan, list subscriptions, and collection numbers by plan for the dashboard.

Test before moving on: each API works from Postman. Register with Free, then register with a paid plan and confirm the payment. Nothing is blocked yet.

5 Frontend: public pages and company admin

- Public pages (outside both dashboards): plans page, register form with plan selection, dummy payment page.
- Company admin: a new **Subscriptions** page (current plan, days left, history, renew, switch to free).
- The "plan expired: choose a plan or start free" screen.

Test before moving on: you can go from the plans page, through register and payment, to the company admin dashboard, for both Free and paid.

6 Frontend: super admin

- Plan management page (edit price, duration, free plan active or inactive).
- Company active or inactive switch.
- Register a company from the super admin side using the same flow.
- Dashboard: total collection by plan, which company has which plan, who is expired.

Test before moving on: changing a plan price does not change any existing company's subscription, and the dashboard numbers match the payment records.

7 Backend part B (turn blocking on)

- Login check for **all roles** (company admin, staff, accountant, vendor, customer): company must be active and have a valid subscription.
- The same check on every company request, so a plan that expires while someone is logged in is also caught. Keep the plans, buy and start-free routes reachable when expired.
- Plan limits: block creating new customers, vendors, staff and invoices past the plan limit. Existing records stay readable.
- Set `ENFORCE_SUBSCRIPTION=true`.

Test before moving on: expire a test company by hand (set its end date to yesterday). Login must redirect to the expired screen, and buying a plan must bring it back.

8 Final regression test

- Test the full journey for Free, Monthly and Yearly.
- Test suspended company, expired company, and a company over the Free limit.
- Confirm the existing company admin and super admin flows still work.

Test before moving on: nothing that worked before the subscription work is broken.

Note: the daily background job (marking expired plans automatically) is skipped in the MVP. Instead, compare the end date with today at login and in the super admin numbers, so nothing has to run in the background.

2. How to build the dummy payment

The goal is a fake payment that behaves like a real one, so that later you replace **one file** and nothing else changes.

The flow in plain words

- The user picks a paid plan and submits the register form.
- The backend creates the company as `PENDING_PAYMENT`, a subscription as `PENDING`, and a payment record as `PENDING`.
- The user lands on the dummy payment page and clicks **Pay now** (or **Simulate failure**).
- On success the backend marks the payment PAID, the subscription ACTIVE (start = today, end = today + plan duration) and the company ACTIVE. On failure the payment is FAILED and the company stays pending.
- The user is sent to login.

Rules to follow

- **One file for payment logic.** Create `paymentService.js` with two functions: `createPayment(subscription)` and `confirmPayment(paymentId)`. Right now they are fake. Later you swap only this file for Razorpay or Stripe.
- **Use PAYMENT_MODE=dummy.** The dummy confirm endpoint must work only when this is set to dummy, so it cannot stay open by accident in real use.
- **Dummy page with two buttons.** Show a simple fake form with **Pay now** and **Simulate failure**, so you can test both outcomes.
- **Never store card numbers**, even fake ones. Keep only amount, status and method `DUMMY`.
- **Price always comes from the database**, never from what the frontend sends.
- **Confirming twice must be safe.** If the payment is already PAID, just return OK. Do not extend the plan again.

Pseudo code

```
confirmPayment(paymentId):
  payment = find payment
  if payment.status == PAID -> return ok      (safe to run twice)
  if PAYMENT_MODE != dummy  -> reject
  transaction:
    payment      -> PAID, paidAt = now, method = DUMMY
    subscription -> ACTIVE
                    startDate = now
                    endDate   = now + plan.durationMonths
    company      -> ACTIVE
    (renewal) old subscription -> CANCELLED
```

Dummy now versus real later

	Dummy (now)	Real gateway (later)
Who confirms payment	Your own confirm button and API	The gateway, checked by signature or webhook
Files to change	None	Only `paymentService.js` plus the payment page
Extras needed	Nothing	API keys, webhook URL, signature check
Payment method saved	DUMMY	The gateway name

3. What you may have forgotten (MVP only)

Read these before you start coding. Items 4 to 7 are decisions, and the table below shows my simple defaults.

Before you start

1. **Back up the database**
   The migration changes tables. A backup lets you undo a mistake in one minute.
2. **Seed the plans**
   Create Free, Monthly and Yearly with price, duration and limits. Without plans the plans page and register form have nothing to show.
3. **Give every existing company a subscription**
   Your current companies have no subscription. Run a one-time script that gives each one (for example) the Free plan. Without it, the login check will lock everyone out the day you turn it on.

Decisions to make now

#	Decision	Simple default	What it means
4	Free plan limits	You choose the numbers	Maximum customers, vendors, staff and invoices on Free. Paid plans have no limit.
5	Upgrade or renewal	Remaining days are lost	Buying a new plan starts a fresh period from today. Easiest to build.
6	Going back to Free	Yes, always allowed	An expired company can always start Free instead of paying.
7	Over the limit on Free	Block new records only	Existing data is kept and readable. Only creating more is blocked.

While building

8. Enforcement switch

Keep an on/off setting such as `ENFORCE_SUBSCRIPTION`. Build and test every screen with it off, then turn it on last.

9. Expired-plan screen access

When a plan is expired, the company admin must still reach three things: view plans, buy a plan, and start Free. Do not block those routes.

10. Double-click safety

If someone clicks Pay twice or refreshes, it must not create two subscriptions or add time twice. Check the payment status first.

11. Pending companies

A company created before payment stays PENDING_PAYMENT if the user leaves. It also keeps that email and slug reserved. For the MVP, give the super admin a way to see and delete pending companies, or clean the old one up when the same email registers again.

12. Public pages

The plans page, register form and payment page belong to neither dashboard. Give them their own routes outside the Company Admin and Super Admin areas.

13. Super admin registering a company

It uses the same flow, but the super admin can mark the payment as MANUAL (for example cash received) and skip the payment page.

14. Check expiry using the end date

Skip the daily background job for now. At login and in the super admin numbers, compare the end date with today. A company is expired if its end date has passed.

15. Test after every step

After each step, open the existing company admin and super admin pages to be sure nothing that worked before has broken.

Skipped until the industry-level phase

Real payment gateway, webhook and signature check, receipts or invoices for subscription payments, expiry reminder emails, refunds, proration, and the daily background job.